

Encrypted Diary

A Privacy-Focused Digital Journaling Application

Presented by: MONIKA G K

Advanced Python Programming Project

THE PROBLEM

Private thoughts need more than account access

A familiar login does not automatically create a separate privacy boundary for every diary.



Personal by nature

Digital journals hold personal thoughts, experiences, and private notes.



One account layer

An account password controls access to the account, but may not separately protect each diary's content.



Exposure matters

Unauthorized access to stored data can expose private information.



Separate spaces

Users may want different private spaces with separate passwords.

How can journaling stay simple while diary content gets an additional layer of protection?

THE SOLUTION

Encrypted Diary adds a privacy layer to familiar journaling

The core experience stays recognizable: register, log in, create a diary, and write.

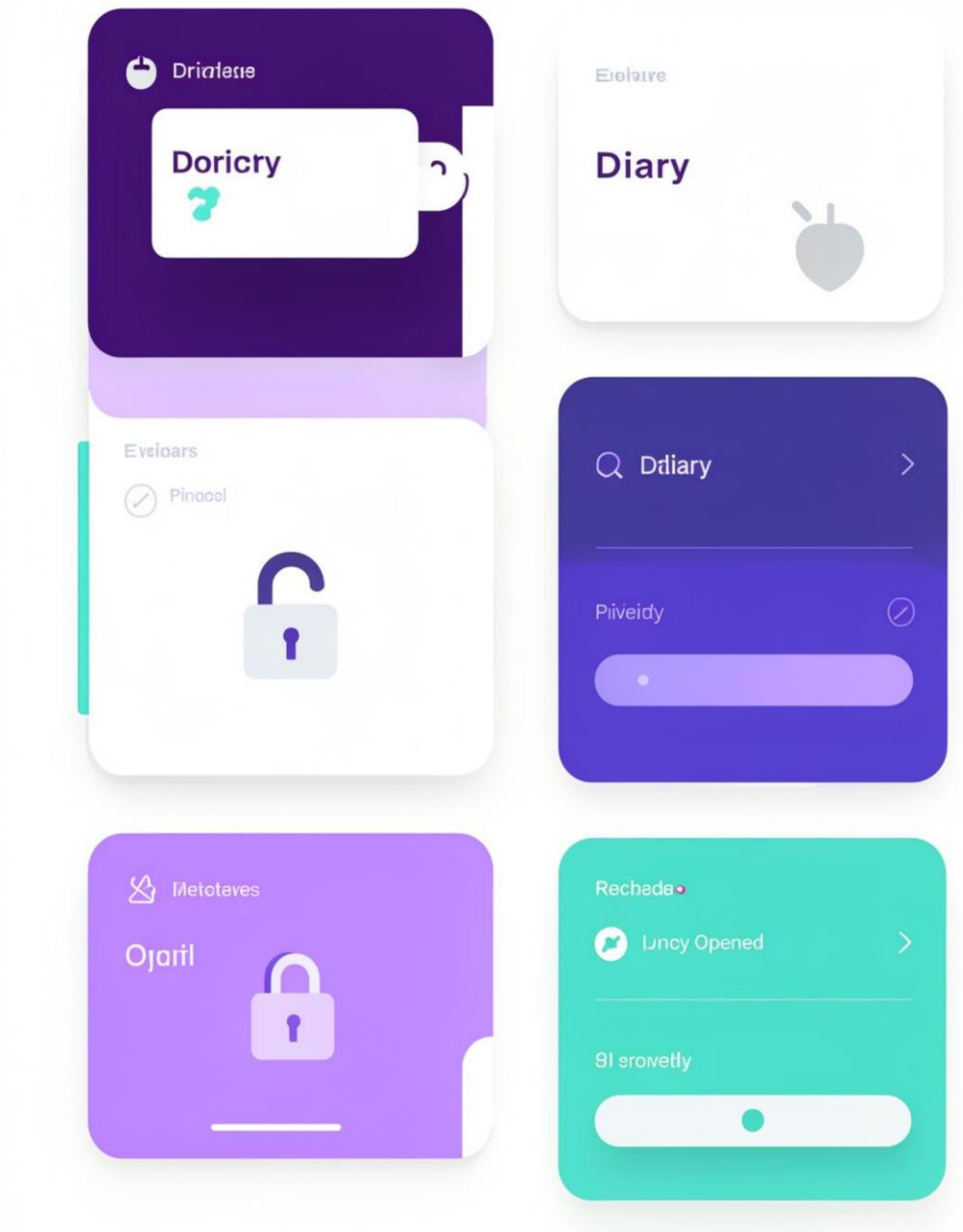
01 Multiple diaries for each user, each with its own password.

02 Entry titles and contents are encrypted before database storage.

03 Password-based unlocking decrypts content for authorized viewing.

04 Create, view, delete, lock, and logout actions support the core flow.

Value proposition: convenient journaling with additional privacy through encrypted diary content.



WHO IT SERVES

One account, separate private spaces

Students

Maintain private journals.

Professionals

Store personal reflections.

Privacy-conscious users

Control access to digital notes.

Flexible organization

Keep different parts of life in separate diaries.

VALUE PROPOSITION

User need → Encrypted Diary → Privacy + Convenience



CORE WORKFLOW

From login to a locked, readable diary

The application makes the secure path visible: unlock first, then read or write.

01

Register or log in

Start a session.

02

Open dashboard

Choose a diary.

03

Create or select

Use a separate password.

04

Enter password

Flask verifies it.

05

Derive key

Build the encryption key.

06

Unlock diary

Decrypt entries for viewing.

07

Work with entries

Create, read, or delete.

08

Save encrypted

Store in PostgreSQL.

09

Lock or log out

End access.

Incorrect password → error message → diary remains locked.

Enchanted Pages
Application Architecture

A secure diary platform with encrypted storage and Telegram integration

Client (User)

Web Browser
(Desktop / Mobile)



- Register / Login
- Manage diaries
- Write / edit / view entries
- Generate Telegram link code
- Import Telegram entries

Telegram App



- Send commants (/start, /link, /save, /saved, /unlink)
- Receive bot responses
- Send messages to save as diary entries

HTTPS Requests (Frontend UI)

HTTPS Webhook (Updates)

Bot Responses (via Telegram API)

Vercel (Cloud Deployment)

Flask Application
(Backend + Web Server)

Web Routes

- User authentication
- Diary management
- Entry CRUD operations
- Generate Telegram link code
- Import Telegram entries
- Dashboard and UI rendering

Telegram Webhook

- Receives Telegram updates
- Processes bot commands
- Validates link codes
- Stores Telegram messages as diary entries
- Sends responses to users

Core Modules

Encryption (Fernet + PBKDF2)
Encrypts/decrypts entry titles and content using diary password

Session Management
User sessions and diary unlock sessions

Database Layer
Database connection, queries and operations

Utilities & Services
Link code generation, validation and Telegram integration

Database Operations (Read / Write)

Database (Neon PostgreSQL)

Stores application data

- Users
- Diaries
- Entries (encrypted)
- Unlock sessions
- Telegram links
- Telegram saved messages

HTTPS (API Calls)

Telegram Bot API
(Telegram Servers)

- Receives webhook URL
- Sends user messages and updates to the application
- Delivers bot responses to users

Key Data Flow

1. **User Action**
   User performs an action on the web app or sends a command on Telegram.
2. **Request to Server**
   Request is sent securely via HTTPS to the Flask application on Vercel.
3. **Authentication & Processing**
   Application authenticates the user, processes the request, and uses encryption and database operations as needed.
4. **Encrypted Storage**
   Diary entry title and content are encrypted and stored in PostgreSQL.
5. **Response to User**
   Application sends the appropriate response back to the web browser or via Telegram Bot API.

SYSTEM ARCHITECTURE

Three layers keep the application understandable



User's Browser

HTML, CSS, JavaScript, and Flask-rendered Jinja templates.



Flask Application — Python Backend

Authentication, sessions, diary and entry management, password verification, encryption, and decryption.



PostgreSQL on Neon

Users, diaries, entries, and unlock_sessions.



Browser ↔ HTTP requests and responses ↔ Flask



Flask → SQL queries through psycopg2 → PostgreSQL



GitHub source code → Vercel deployed Flask application



Vercel application → Neon hosted PostgreSQL database

SECURITY MODEL

A diary password becomes a protected encryption key

01

Password + salt

Each diary uses a unique salt.

02

PBKDF2-HMAC-SHA256

Derives the diary encryption key.

03

Fernet encryption

Turns titles and contents into ciphertext.

04

Authorized viewing

Verify → derive → decrypt → read.

05

Limited unlock session

Stores a protected copy of the derived key with limited expiry.

Hashing

Account passwords are hashed with Werkzeug password-hashing functions. Hashes are for verification, not recovery.

Encryption

Entry titles and contents are encrypted so authorized viewing can decrypt them.

Security boundary: entry content is protected, but not all metadata is encrypted. This is not a fully zero-knowledge or end-to-end encrypted system.

ENCHANTED PAGES — PYTHON MODULE WORKFLOW

From user action to encrypted storage and secure reading

A secure space for your thoughts and memories

1 User & Browser



- Register
- Login
- Dashboard
- Create / Open diary
- Write and manage entries

HTTP request

2 app.py | Flask Controller

- Handles all routes
- Form validation
- Authentication (user & diary)
- Manages unlock sessions
- Coordinates crypto, database and Telegram modules

Telegram workflow

6 telegram_integration.py | Memory Bridge

- Processes bot commands
- /link /save /saved /unlink
- Encrypts staged memories
- Stores in Telegram tables
- Allows import to selected diary

Webhook + API calls

7 Telegram Bot / API

- Receives bot commands
- Sends responses
- Webhook integration
- /link /save /saved /unlink

Encryption / Decryption

Database operations (read / write)

3 crypto.py | Encryption Engine

- PBKDF2-HMAC-SHA256 key derivation
- Fernet encryption / decryption
- Protects unlock keys (using master key)
- Used by app.py for secure processing of diary data

Encrypted data

4 database.py | PostgreSQL Data Layer

- Database connection (psycopg2)
- SQL queries (fetch_one, fetch_all, execute)
- Table initialization (init_db)
- Reads and writes records
- Used by app.py and telegram_integration.py

Staged memories (encrypted)

Import selected entries to diary

Store / Retrieve encrypted records

5 PostgreSQL Database

- users
- diaries
- entries (encrypted title & content)
- unlock_sessions
- telegram_links
- telegram_saved

ENTRY LIFECYCLE

01 AUTHENTICATE

User logs in and creates/selects a diary. Diary password is verified to access the diary.

02 ENCRYPT & SAVE

Title and content are encrypted with Fernet using the diary key before insertion into PostgreSQL.

03 UNLOCK & READ

A valid unlock session permits ciphertext decryption and the original content is displayed in a readable form.

EVALUATION

Strong foundation, honest next steps

Current benefits

- Encrypted stored content
- Separate diary passwords
- Multiple diaries and entries
- Relational storage
- Simple browser access

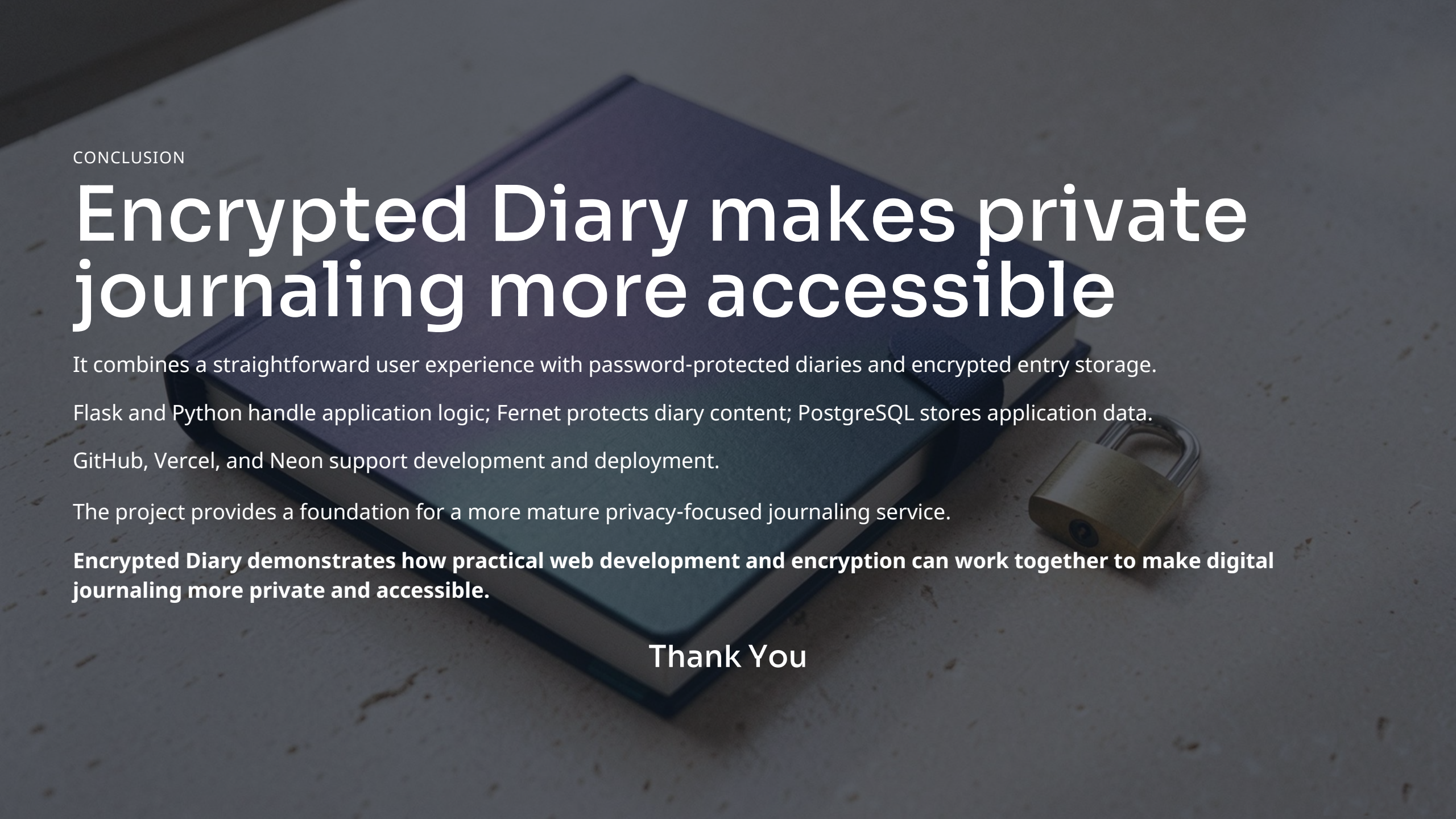
Current limitations

- Not fully zero-knowledge
- Some metadata remains readable
- Recovery design needs care
- More security and usability testing is needed before production use

Planned or potential

- Secure backup and recovery
- Editing, search, and reminders
- Analytics and mobile support
- Stronger sessions, rate limiting, and audits
- Possible subscription plans

Future, planned, or potential items are clearly separate from the current application.



CONCLUSION

Encrypted Diary makes private journaling more accessible

It combines a straightforward user experience with password-protected diaries and encrypted entry storage.

Flask and Python handle application logic; Fernet protects diary content; PostgreSQL stores application data.

GitHub, Vercel, and Neon support development and deployment.

The project provides a foundation for a more mature privacy-focused journaling service.

Encrypted Diary demonstrates how practical web development and encryption can work together to make digital journaling more private and accessible.

Thank You